ECSE-6320 Advanced Computer Systems
Project 3 Report

Cooper Werner
Rensselaer Polytechnic Institute
wernec6@rpi.edu

October 23, 2026

Contents

1 Introduction **2**

2 Pre-Experiment Hypotheses **2**

3 Measurement Discipline **3**
3.1 Experimental Setup . . . 3
3.2 Statistical Treatment . . . 3

4 Results **4**
4.1 Bandwidth vs Latency for Various Queue Depths . . . 4
4.2 Zero-Queue Baseline Latency and Bandwidth . . . 4
4.3 Near-Knee Tail Latency Characterization . . . 5
4.4 Block Size Sweep . . . 5
4.4.1 Random Read / Write . . . 5
4.4.2 Sequential Read / Write . . . 6
4.5 Read / Write Mix Sweep . . . 7
4.6 Impact of Data Compressibility On Latency . . . 8
4.7 Impacts of SSD State on Latency and Bandwidth . . . 8

5 Negative / Failure Case **9**

6 Discussion **9**

7 Conclusion **9**

A AI Usage **9**

B Additional Data / Raw Logs **10**

1 Introduction

The goal of this project is to analyze how block size, access pattern, read/write ratio, queue depth, data pattern, and device state impact SSD performance. I utilized the FIO utility to run tests on my secondary SSD, and I report my findings below.

2 Pre-Experiment Hypotheses

Pre-Experiment Hypothesis

Hypothesis:

I think that in general, SSD writes will be slower than reads utilizing the same block size, access pattern, and queue depth.

Reasoning:

From my prior experience installing software and copying files, I have observed that writing to an SSD is slower than reading from it. So I would expect that this would be the case in general.

Predicted quantitative outcome:

Table 1: Predicted Read vs Write Speed for Various Block Sizes

Block Size	Read Speed	Write Speed
4 KiB	500 MB/s	400 MB/s
16 KiB	600 MB/s	500 MB/s
32 KiB	700 MB/s	600 MB/s
64 KiB	800 MB/s	700 MB/s
128 KiB	900 MB/s	800 MB/s
256 KiB	1000 MB/s	900 MB/s
512 KiB	1100 MB/s	1000 MB/s

Pre-Experiment Hypothesis

Hypothesis:

Accessing data with in a sequential pattern will be faster than accessing data with a random pattern, given the same block size and queue depth.

Reasoning:

Sequential access patterns allow the SSD to optimize its internal operations, such as read-ahead and write buffering, leading to better performance compared to random access patterns. It will also reduce the number of seek operations and improve cache utilization, resulting in lower latency and higher throughput.

Predicted quantitative outcome:

Table 2: Predicted Slowdown for Sequential vs Random Access

Access Pattern	Slowdown For Read	Slowdown For Write
Sequential	1.0x	1.0x
Random	1.5x	1.5x

3 Measurement Discipline

To collect the data I utilized AI to help me write a shell script tha ran a variety of FIO commands to collect data about my SSD's performance. To ensure the safety of my data, I ensured that the script required the user to enter the specific drive they are profiling, the script will refuse to run if the drive has a root or boot directory on it. Then the user has to confirm execution before testing on the drive will start.

3.1 Experimental Setup

- **Hardware:** CPU: x86_64 AMD Ryzen AI 7 350 w/ Radeon 860M, Memory: 32GB LPDDR5 @ 5600MT/s, SSD: OEM Samsung SSD 980 Pro 1TB NVMe PCIe Gen4
- **OS / Kernel:** Pop_OS! 24.04 Running Linux 7.0.11-76070011-generic
- **FIO Version:** fio-3.36
- **Isolation techniques:** Tested on dedicated drive with no other processes running, CPU frequency scaling disabled, and CPU pinned to a single core.
- **GB Written Per Trial:** 10 GB for each test to ensure steady-state performance.

The computer was restarted before the test to ensure that the SSD was in a known state, the comptuer was idle, and no other processes were running. The computer was plugged in and running on AC power to avoid any performance throttling due to battery saving features. Lastly, the CPU govenor was set to performance mode to avoid any frequency scaling during the tests.

3.2 Statistical Treatment

Due to the large number of experiments and the time required to run each one, I only ran each test once. However, I ensured that each test was run for a sufficient duration (10 GB of data written) to reach steady-state performance, minimizing the impact of transient effects on the results.

4 Results

4.1 Bandwidth vs Latency for Various Queue Depths

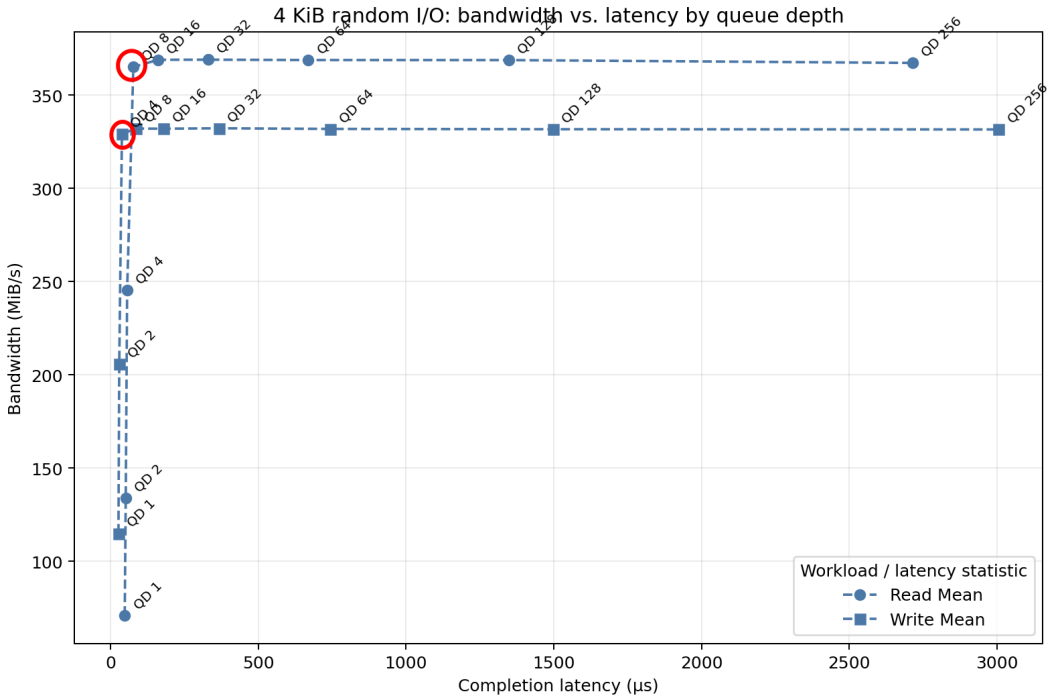


Figure 1: Plot of bandwidth vs latency for various queue depths. At a queue depth around 4 or 8, bandwidth capacity increases significantly, and the latency of each request starts to increase rapidly.

Figure 1demonstrates that SSDs can optimize for bandwidth or latency using request queuing. Having a greater queue depth means that the SSD can best utilize the PCIe bus for transfering data to/from RAM. This queuing comes with a tradeoff for latency, as queue depth grows, latency also does as predicted by Little's Law: $L = \lambda R$. Increasing the queue depth increases the number of outstanding requests, L, and since λ is a constant, the latency for each individual request must increase too.

4.2 Zero-Queue Baseline Latency and Bandwidth

Table 3: Zero-Queue Baseline Latency and Bandwidth

Setup	Mean BW (MB/s)	P95 Lat. (μs)	P99 Lat. (μs)	P99.9 Lat. (μs)
4KiB Random Read	67.9	63.7	74.4	150.5
4KiB Random Write	118.2	37.1	43.3	81.4
128KiB Sequential Read	1,490.7	116.2	144.4	199.7
128KiB Sequential Write	1,755.7	82.4	119.3	173.1

The zero-queue baseline latency and bandwidth data from table 3 show an interesting pattern that shows up a few times in the data, the write bandwidth is larger than the read bandwidth and the

write latency is lower than the read bandwidth. This is true for both access patterns and block sizes for the baseline. This contradicts my hypothesis from above.

4.3 Near-Knee Tail Latency Characterization

Table 4: Near-Knee Tail Read Latency Characterization

Queue Depth	P95 Lat. (μs)	P99 Lat. (μs)	P99.9 Lat. (μs)
QD4	82.4	100.8	130.6
QD8	105.9	128.5	152.5
QD16	181.2	203.8	236.5

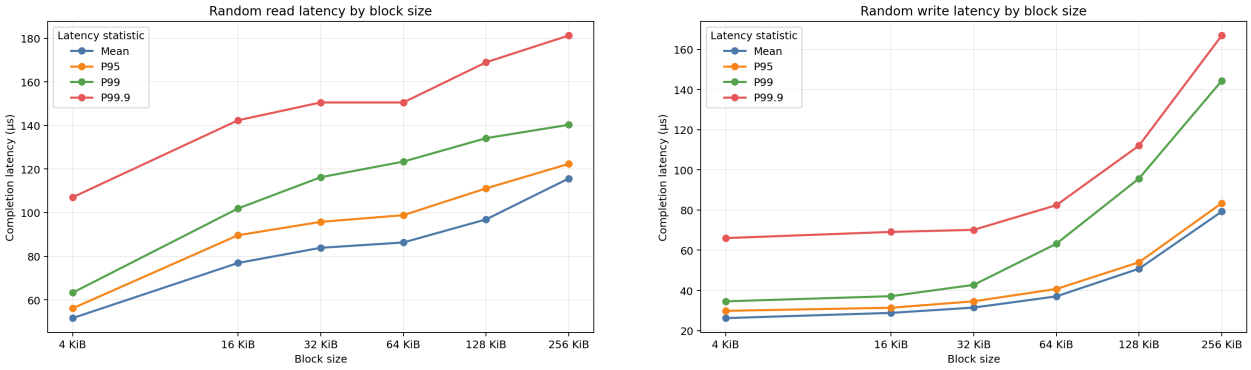
Table 5: Near-Knee Tail Write Latency Characterization

Queue Depth	P95 Lat. (μs)	P99 Lat. (μs)	P99.9 Lat. (μs)
QD4	55.0	70.1	85.5
QD8	107.0	130.5	152.6
QD16	214.0	220.1	276.5

As seen in Figure 1 the knee queue depth for read is a queue depth of 8, for write the knee is at a queue depth of 4. By looking at the tail latencies, for read, having a queue depth of 4 does not significantly decrease the latency of each request while significantly impacting the maximum bandwidth.

4.4 Block Size Sweep

4.4.1 Random Read / Write

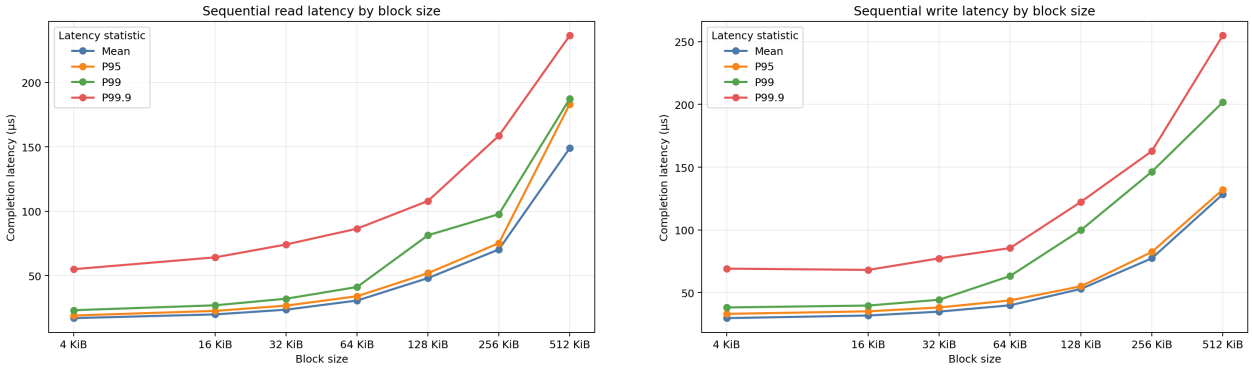


(a) Plot of SSD random read latencies for different sized blocks with a fixed queue depth of 1.

(b) Plot of SSD random write latencies for different sized blocks with a fixed queue depth of 1.

Figure 2: Side-by-side comparison of ...

4.4.2 Sequential Read / Write



(a) Plot of SSD sequential read latencies for different sized blocks with a fixed queue depth of 1.

(b) Plot of SSD sequential write latencies for different sized blocks with a fixed queue depth of 1.

Figure 3: Side-by-side comparison of ...

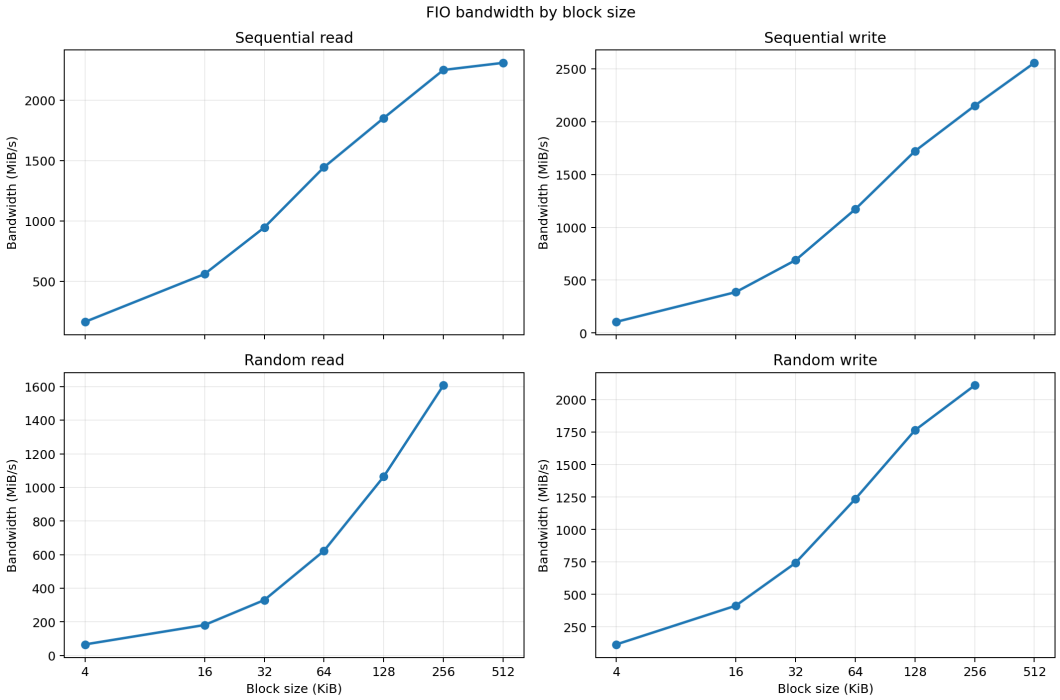


Figure 4: Plots of bandwidth as block size changes with a fixed queue depth of 1.

Changing the size of the blocks has a very similar effect to changing the queue depth. As you increase block size, the SSD is able to batch requests and read / write more to the NAND flash cells. However this comes at the cost of latency, larger blocks take more time to write to the drive and therefore will have larger latencies.

4.5 Read / Write Mix Sweep

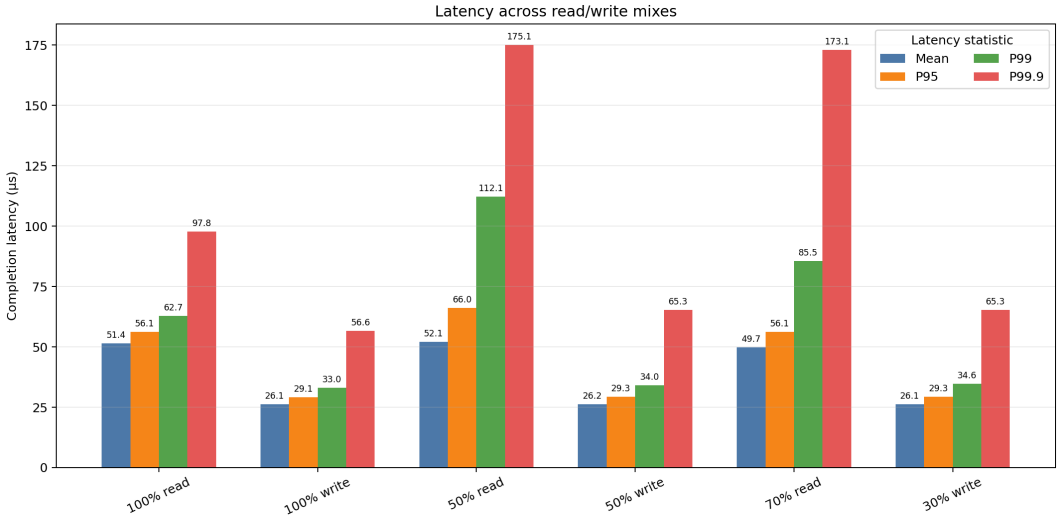


Figure 5: Bar chart showing the average, P95, P99, and P99.9 latencies for read and write operations for the different mixes. The mixes tested were 100% read, 100% write, 50/50% read-write, 70%/30% read-write.

The read operations had latencies that were consistently higher than the write latencies. In the mixes with 50% or 70% reads the P99 and P99.9 latencies for the read operations specifically increased far past the mix where reads were 100% of the operations. This did not happen for the write operations which had consistent latencies for each of the mixes. This may be due to the SLC write cache helping to cache write requests while reads are in progress, but the read operations have to be blocked if a write operation is happening at the same time since data served must be up to date.

4.6 Impact of Data Compressibility On Latency

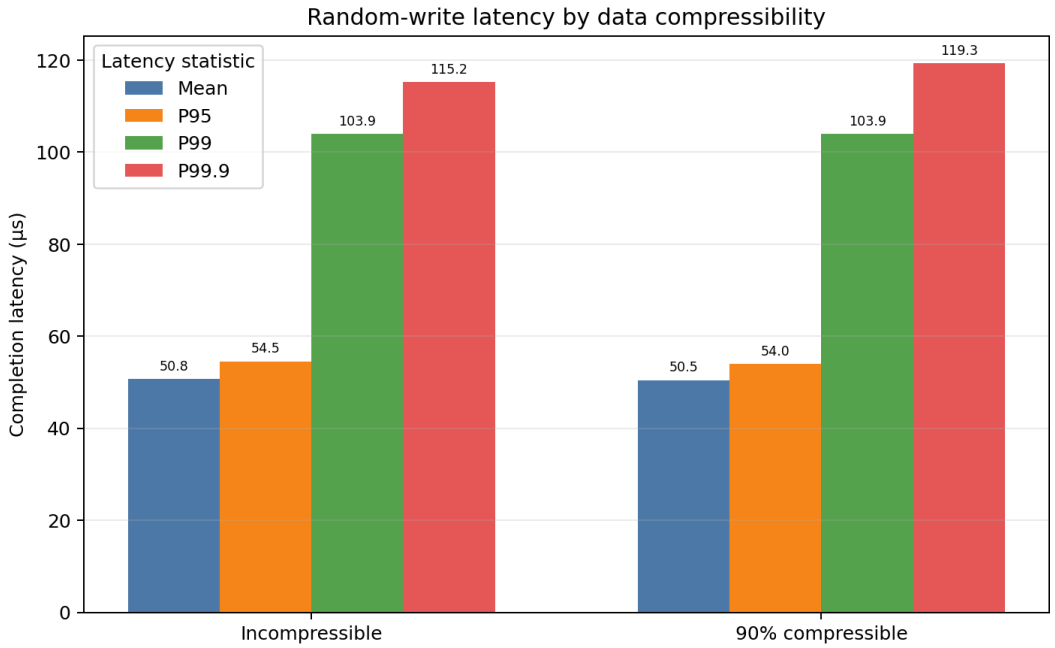


Figure 6: Bar chart showing latencies for saving incompressible data and highly compressible data.

There was no significant difference in lattency for saving compressible vs incompressible data. This is because the SSD tested, the SAMSUNG MZVL21T0HCLR, lacks a hardware-level data compression function in the SSD controller. This means that regardless of data compressibility, the same amount of data is written to disk and the latencies remain the same. [1][2]

4.7 Impacts of SSD State on Latency and Bandwidth

Table 6: Impacts of SSD State on Write Latency and Bandwidth

SSD State	Mean BW (MB/s)	P95 Lat. (μs)	P99 Lat. (μs)	P99.9 Lat. (μs)
Fresh First Write	1,748.7	57.6	99.8	124.4
Garbage Collection Stress	314.7	432.1	1,581.1	2,736.1
Garbage Collection Recovery	327.6	415.7	1,028.1	2,039.8
Steady State	3,232.2	2,146.3	3,915.8	6,651.9
SLC Cache Exhaustion	3,473.3	7,634.9	8,454.1	13,434.9

Table 6 demonstrates the variablility in SSD performance depending on SSD state. When the SSD is fresh, it is able to maintain a high write bandwidth with low latencies. As the SSD fills up and the garbage collector starts running the maximum write bandwidth decreases significantly and latencies start increasing. In steady-state the drive has an impressive write bandwidth, but its latencies can increase to close to 6ms for a single request. This is amplified when the SLC cache is exhausted, cache exhaustion increases the P99.9 latencies to over 13ms.

5 Negative / Failure Case

Negative / Failure Case

Experimental Pitfall:

My script runs the FIO commands immediately after the termination of the previous one without providing the SSD time to rest, handle garbage collection, or cool down. This could create confounding results in the different experiments. I also don't have good data on the SSD temperature during the experimental runs. Lastly, my SSD lives in a laptop chassis which does not provide an optimal path for heat dissipation.

How would I fix this?

If I were to continue looking at this I would change the experimental setup to give the SSD time to return to restful state between each run. I would also move testing from my laptop to a desktop tower or open-air testbench to remove the impacts of thermal effects.

6 Discussion

My first hypothesis was disproven by the data I collected. Consistently, the write operations had higher bandwidths and lower latencies than read operations of the same queue depth, block size, and access pattern. This is a bit perplexing to me give my experience installing software and copying files between disks in the past where read operations feel faster than write operations. Perhaps this is due to the SLC cache helping to make writes more efficicent and that my test file size was not large enough to fully exhaust the cache. It would be interesting to look into this further.

My second hypothesis was mostly correct, but the slowdown was not as extreme as I had predicted it to be. Random reads had a 1.43x decrease in bandwidth, but only a 1.09x increase in latency. The general idea was correct, but my predicted magnitude was off by a bit.

I noticed that for the bandwidth vs block size plots, the trend indicated that increasing the block size futher would be able to increase bandwidth further. It would be interesting to explore this further and look at which queue depths and block sizes maximize bandwidth or minimize latency.

7 Conclusion

SSD Performance is strongly dependent on block size, access pattern, queue depth, and device state. If latency or bandwidth is important for the application, programmers should carefully balance block size, queue depth, and access pattern to ensure that SSD characteristics match the required spec for the application.

A AI Usage

AI helped me write the code required for the experiment, it helped develop scripts for running all the required experiments, assisted in developing python code to create plots and figures. Lastly, I utilized AI in developing the experimental design and troubleshooting anomalies in the data.

1. Helped me determine the best way to perform this experiment: https://claude.ai/share/64d5d392-8521-4067-9c41-80704ce5ede0

I also made extensive use of OpenCode (an open source coding agent) to develop scripts, experiment code, and plotting code. I reveiewed all output for correctness and accuracy. Making minor fixes where applicable. Transcripts for the OpenCode sessions are available in `./OpenCode`

B Additional Data / Raw Logs

Raw data can be found `./fio-results-20261016-200949`. Plots and plotting code can be `./analysis`.

References

[1] M. Chiappetta, "Samsung ssd 980 pro," HotHardware, 9 2020, accessed 2026-10-23. [Online]. Available: https://hothardware.com/reviews/samsung-ssd-980-pro-nvme-pcie-4-review

[2] Klaus Hinum, "Samsung pm9a1 mzvl21t0hclr ssd benchmarks," Notebookcheck, 5 2022, accessed 2026-10-23. [Online]. Available: https://www.notebookcheck.net/Samsung-PM9A1-MZVL21T0HCLR-SSD-Benchmarks.623658.0.html